COMSAT University Islamabad Sahiwal Campus

(Department of SE)

Object Oriented Programming Project

Smart Semester Progress Tracker

By

Maryam Zahid (FA24-BSE-035-A)

Abdullah Tahir (FA24-BSE-026-A)

Muhammad Danish (FA24-BSE-047-A)

Instructor: Mr. Ameer Gillani

Table of Contents

1. Introduction 4

2.1 Requirement Analysis 5

2.2 System Design 5

2.3 User Interface Development 6

2.4 Implementation 7

2.5 Data Handling and Storage 7

2.6 Testing and Debugging 8

2.7 User Feedback and Improvement 8

2.8 Final Deployment 8

3. UML DIAGRAMS 8

3.1 Class Diagram 9

3.2 Use Case Diagram 9

3.3 Sequence Diagram 11

3.4 Activity Diagram 12

3.5 State Diagram 13

4. PROJECT FLOW 14

4.1 Application Initialization Flow 14

4.2 Authentication and Login Flow 15

4.2.1 **Student Login Flow:** 15

4.2.2 **Admin Login Flow:** 15

4.3 Student Academic Management Flow 15

4.4 GPA Calculation and Academic Evaluation Flow 15

4.5 Report Generation, Saving, and Printing Flow 16

4.6 Admin Monitoring Flow 16

4.7 Data Storage and Retrieval Flow 16

4.8 Logout and Application Termination Flow 16

5. GRAPHICAL USER INTERFACE (GUI) DESCRIPTION 17

5.1 Welcome / Main Screen 17

5.3 Student Registration Screen 18

5.2 Student Login Screen 19

5.4 Student Dashboard Screen 20

5.5 Course Registration (Add New Course) Screen 20

5.6 Update Course Details Screen 21

5.7 Delete Course Screen 22

5.8 Attendance Entry Screen 23

5.9 Student Profile and Course Overview Screen 24
5.10 Marks Entry and GPA Calculation Screen 25
5.11 Detailed Result Screen 26
5.12 GPA Calculation Result Screen 28
5.13 Semester Result Card Screen 29
5.14 Report Generation and PDF Export 30
5.15 Administrator Login Screen 32
5.16 Admin Control Panel (Admin Dashboard) 32
6. Database Queries and Tables 33
6.1 Student Table and Query 33
6.2 Course Table and Query 34
6.2.1 Insert Table and Query 34
6.3 Course Registration Table and Query 35
6.4 Attendance Table and Query 36
6.5 Marks Table and Query 37
7. Team Contribution Table Smart Semester Progress Tracker 38
7.1 Smart Semester Progress TrackerTable For Marks 39

1. Introduction

In modern academic environments, students are required to manage multiple courses, attendance records, quizzes, assignments, lab work, midterm examinations, and final assessments simultaneously. Tracking all these academic components manually often leads to confusion, inaccurate calculations, misplaced records, and poor visibility of overall semester performance. As academic pressure increases, students need a reliable and automated system to monitor their progress efficiently.

The **Smart Semester Progress Tracker** is a desktop-based academic management system designed to provide students with a centralized and intelligent platform for tracking their semester performance. The system is developed using **Java** with a **Java Swing-based graphical user interface** and is integrated with a **MySQL database server** to ensure secure, structured, and permanent data storage. The application focuses on simplifying academic data management while maintaining accuracy, consistency, and ease of use.

The system supports **role-based access**, providing separate interfaces for **Students** and **Administrators**. Students can register courses, manage attendance records, enter marks for quizzes, assignments, and exams, and view calculated GPA and progress reports. The Administrator panel provides complete visibility of student records stored in the database, allowing academic monitoring and system supervision without compromising data integrity.

A major strength of the Smart Semester Progress Tracker is its **user-friendly and visually modern GUI design**, which ensures smooth navigation across all modules. Each academic function is implemented as a separate screen, reducing complexity and improving usability. Proper input validation, exception handling, and database connectivity mechanisms are implemented to ensure system reliability and data security.

This project demonstrates the practical application of **Object-Oriented Programming (OOP) concepts**, **database-driven application development**, and **software engineering principles**. By integrating backend logic, graphical user interfaces, and database management into a single cohesive system, the Smart Semester Progress Tracker provides an effective solution for academic progress monitoring in university-level environments.

2. METHODOLOGY

The development of the **Smart Semester Progress Tracker** followed a structured and systematic software engineering methodology to ensure correctness, usability, scalability, and alignment with academic requirements. The project was designed and implemented by carefully analyzing user needs, planning system architecture, developing an interactive graphical user interface, implementing backend logic using object-oriented principles, integrating database storage, and performing thorough testing and validation. Each phase of the methodology played a critical role in delivering a complete and reliable academic management system.

2.1 Requirement Analysis

The first phase of the project focused on understanding the academic challenges faced by university students in managing semester-related data. Students are required to track multiple academic components such as **course registration, attendance records, quizzes, assignments, lab work, midterm exams, final exams, GPA calculation, and performance reports**. Manual handling of this data often leads to errors, inconsistency, and lack of visibility.

Based on this analysis, the system requirements were divided into **functional** and **non-functional** requirements.

Functional requirements included:

- Student authentication and secure login
- Course registration for both theory and lab courses
- Attendance management for each registered course
- Marks entry for quizzes, assignments, midterms, and finals
- Automatic GPA and grade calculation based on defined rules
- Generation of detailed academic reports
- Exporting reports into **PDF and Text file formats**
- Administrator login and system-level data viewing
- Role-based access control (Student and Admin)

Non-functional requirements focused on:

- User-friendly and modern GUI design
- Accurate and consistent GPA calculations
- Secure database storage
- Fast data retrieval
- Maintainability and scalability of the system

This phase ensured that all academic and technical expectations were clearly defined before moving to system design.

2.2 System Design

After finalizing the requirements, the system architecture was designed using a **modular and object-oriented approach**. The application was divided into logical components, where each

module is responsible for a specific academic function. This separation improves code readability, maintainability, and future extensibility.

The system design consists of the following major modules:

- Authentication Module (Student and Admin login)
- Student Dashboard Module
- Course Management Module (Theory & Lab courses)
- Attendance Management Module
- Marks Entry Module
- GPA and Grade Calculation Module
- Report Generation and Export Module
- Admin Monitoring Module
- Database Management Module

The system follows a layered structure where:

- The **GUI layer** handles user interaction
- The **business logic layer** processes academic rules and calculations
- The **database layer** manages persistent data storage

This design ensures loose coupling between components and smooth integration across the system.

2.3 User Interface Development

The graphical user interface of the Smart Semester Progress Tracker was developed using **Java Swing**, providing a visually modern, interactive, and user-friendly experience. Each functionality of the system is represented through a **dedicated screen**, ensuring clarity and ease of navigation.

The GUI includes:

- Welcome screen with Student and Admin panel selection
- Student login and registration screens
- Student dashboard displaying academic overview
- Separate screens for course management, attendance, marks entry, GPA calculation, and reports

- Report generation screen with options for **course-wise, category-wise, and full academic reports**
- Admin login and admin dashboard for viewing registered student records

Consistent color themes, typography, buttons, and layouts were used across all screens to maintain a professional look. Validation checks and error messages guide users during data entry, reducing the chances of incorrect input.

2.4 Implementation

The implementation phase involved converting the system design into a fully functional application using **Java**. Object-Oriented Programming (OOP) principles such as **classes, objects, encapsulation, inheritance, and abstraction** were applied throughout the project.

Separate Java classes were created for core entities such as:

- Student
- Course (Theory and Lab)
- Attendance
- Marks and Assessments
- GPA Calculator
- Report Generator
- Admin

Business logic handles calculations for attendance percentages, weighted marks, grade determination, and GPA computation. The report generation module dynamically compiles academic data and formats it into readable reports. Users can **preview reports on-screen**, **save them as text files**, or **export them as PDF documents**, enabling official documentation and printing.

2.5 Data Handling and Storage

All academic data is stored permanently in a **MySQL database server**, ensuring data consistency, security, and reliability. The database contains structured tables for:

- Students
- Courses
- Attendance records
- Marks and assessments
- GPA and semester results

- User authentication data

JDBC (Java Database Connectivity) is used to establish communication between the Java application and the database. CRUD (Create, Read, Update, Delete) operations are implemented for all major modules. Proper exception handling ensures that database errors do not crash the application and that meaningful error messages are displayed when needed.

2.6 Testing and Debugging

Testing was performed at both **module level** and **system level**. Individual modules such as course registration, attendance entry, marks calculation, GPA generation, and report exporting were tested independently. After that, integration testing ensured smooth communication between the GUI, backend logic, and database.

Different test cases were executed using:

- Valid academic data
- Invalid or incomplete inputs
- Boundary conditions for GPA and attendance

Errors identified during testing were corrected, resulting in improved system stability, accuracy, and performance.

2.7 User Feedback and Improvement

During development, feedback was considered regarding usability, screen layout, and workflow clarity. Minor adjustments were made to improve dashboard organization, report readability, and navigation flow. This iterative refinement enhanced the overall user experience and made the system more intuitive for academic use.

2.8 Final Deployment

After successful testing and validation, the Smart Semester Progress Tracker was finalized as a **desktop-based academic management application**. The system is ready to be used in real academic environments where students need reliable semester tracking and administrators require system-level visibility. The modular architecture allows future enhancements such as advanced analytics, web-based access, or extended reporting features.

3. UML DIAGRAMS

Unified Modeling Language (UML) diagrams are used in the Smart Semester Progress Tracker to visually represent the system’s structure, behavior, and workflow. These diagrams help in understanding how different components of the system interact with each other and how data flows across various modules. UML modeling ensures clarity, proper planning, and alignment with object-oriented design principles.

The following UML diagrams were designed for this project:

3.1 Class Diagram

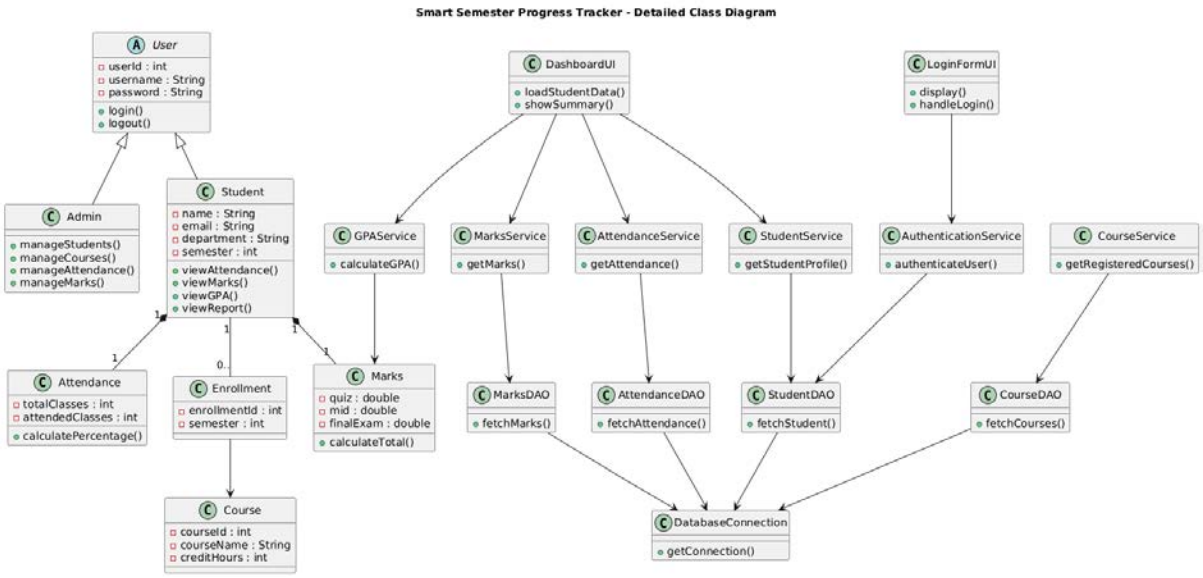
The Class Diagram represents the structural design of the *Smart Semester Progress Tracker* and illustrates the system's layered architecture, including UI, Service, DAO, and Database layers.

The User class acts as a base class and contains common attributes such as user ID, username, and password, along with login and logout functionality. The Student and Admin classes inherit from the User class, demonstrating inheritance and role-based access control.

The Student class stores student-specific academic and personal information and provides functionality to view attendance, marks, GPA, and semester reports. The Admin class allows administrative control such as managing students, courses, attendance, and marks.

The system follows a service-oriented design where business logic is handled by service classes including AuthenticationService, StudentService, AttendanceService, MarksService, GPAService, and CourseService. These services interact with DAO classes (StudentDAO, AttendanceDAO, MarksDAO, and CourseDAO) to retrieve and store data.

The DatabaseConnection class manages connectivity with the My SQL Server database and is used by all DAO classes. This layered design ensures proper encapsulation, separation of concerns, reusability, and maintainability of the system.



3.2 Use Case Diagram

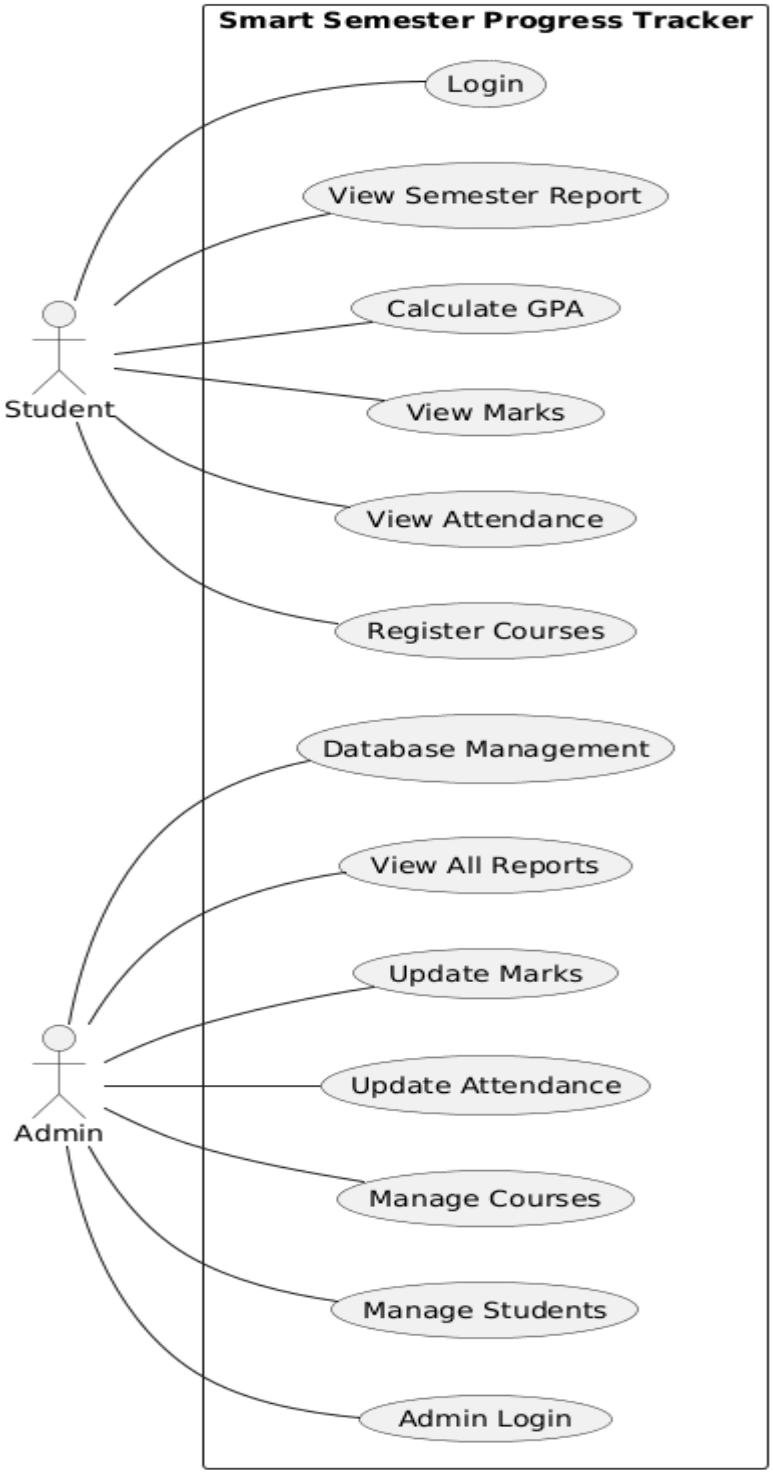
The Use Case Diagram illustrates the functional behavior of the system from the user's perspective and defines the system boundaries and available functionalities.

There are two main actors in the system: **Student** and **Admin**.

The **Student** can log in to the system, register courses, view attendance, view marks, calculate GPA, and view semester reports. These use cases represent the complete academic monitoring workflow of a student.

The **Admin** can log in to the system, manage students, manage courses, update attendance, update marks, view all reports, and perform database management operations. This ensures secure and role-based access to administrative functionalities.

The Use Case Diagram clearly differentiates user responsibilities and enforces controlled interaction with the system.



3.3 Sequence Diagram

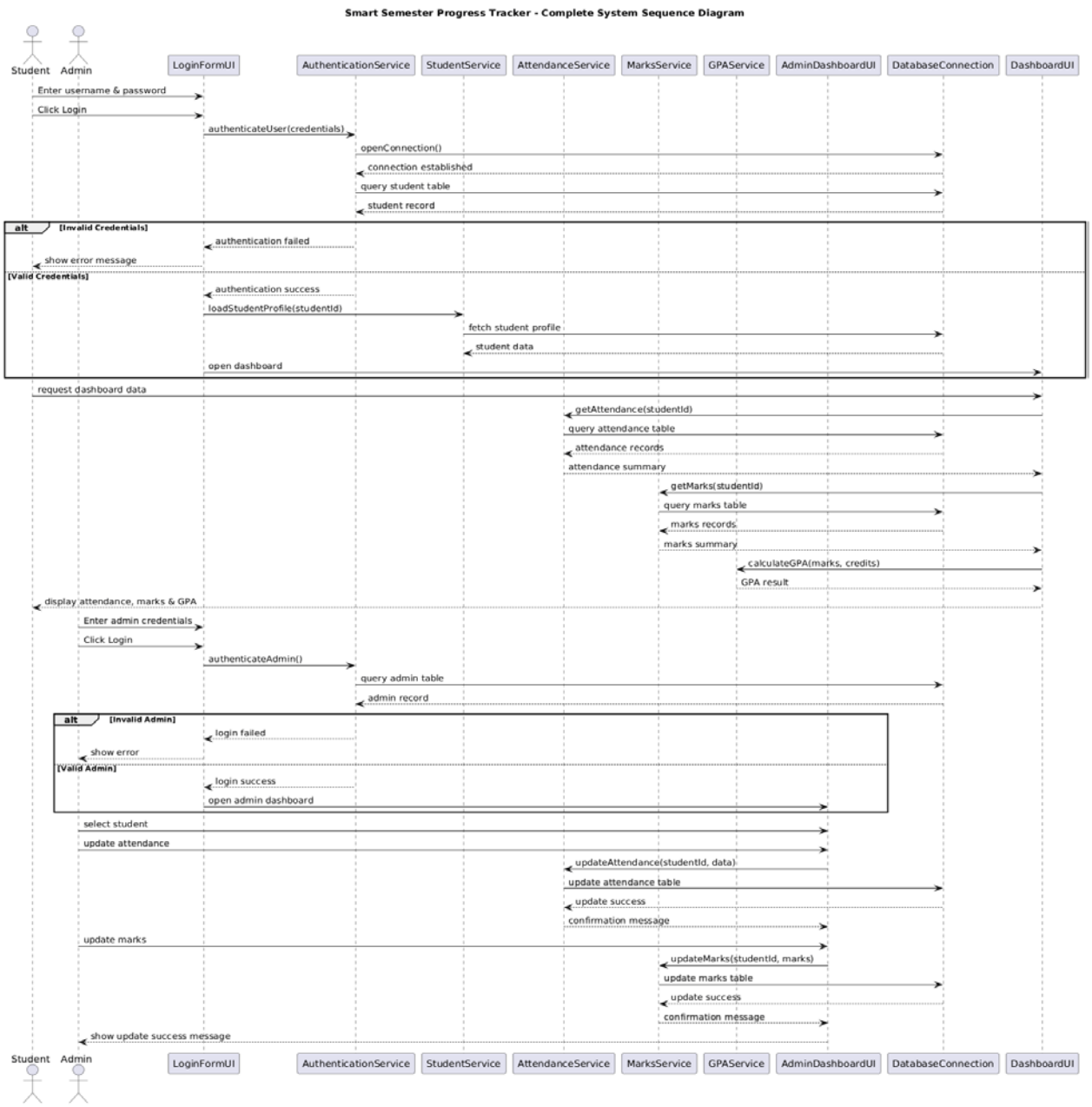
The Sequence Diagram represents the interaction between different system components over time and demonstrates the complete workflow for both student and administrator operations.

The sequence begins with user login through the login interface. The system authenticates credentials using the authentication service and database connection. Upon successful authentication, the student dashboard is loaded, where attendance and marks data are retrieved, GPA is calculated, and the academic summary is displayed.

The diagram also illustrates the administrator workflow, including admin authentication, accessing the admin dashboard, selecting students, and updating attendance and marks. Database updates are confirmed and reflected in the system.

This diagram highlights the order of method calls, data flow, and interaction between UI, ser

vice, DAO, and database layers.



3.4 Activity Diagram

The Activity Diagram represents the complete operational workflow of the *Smart Semester Progress Tracker* for both students and administrators.

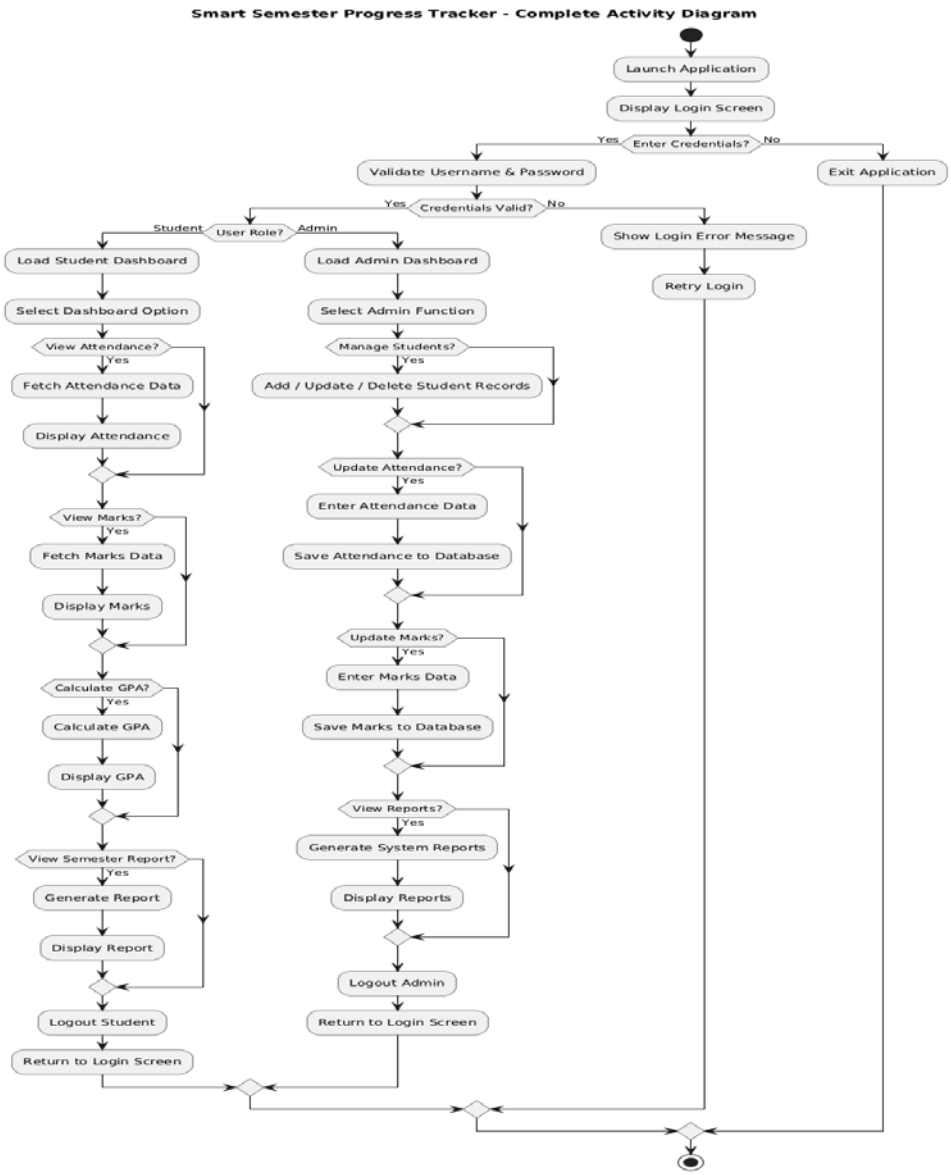
The activity flow starts with launching the application, after which the login screen is displayed. The user enters login credentials, which are validated by the system. If the credentials are invalid, an error message is shown and the user is prompted to retry login or exit the application.

Upon successful authentication, the workflow branches based on the user role. If the user is a **Student**, the system loads the student dashboard. From the dashboard, the

student can view attendance, view marks, calculate GPA, and generate a semester report. Each action involves fetching relevant data from the database and displaying the results. After completing the required tasks, the student logs out and is returned to the login screen.

If the user is an **Admin**, the system loads the admin dashboard. The administrator can manage student records, update attendance, update marks, and view system reports. All updates are saved to the database. Once administrative tasks are completed, the admin logs out and returns to the login screen.

This Activity Diagram clearly illustrates decision points, parallel user roles, and the complete logical flow of the system from start to termination.



3.5 State Diagram

The State Diagram illustrates the different states of the *Smart Semester Progress Tracker* and the transitions between them during system execution.

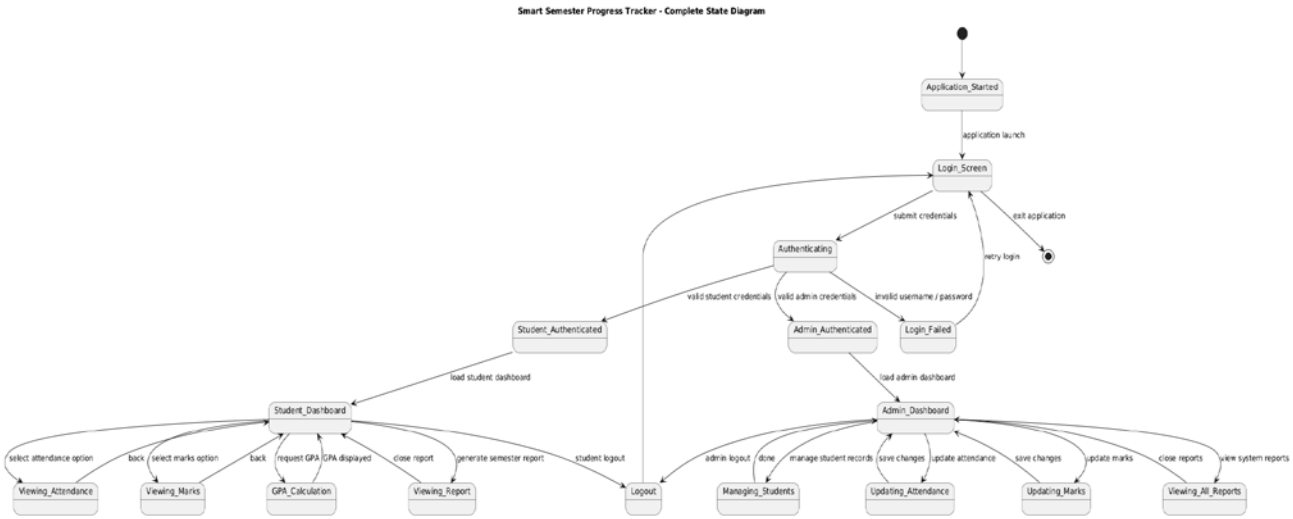
The system begins in the **Application Started** state when the application is launched. It then transitions to the **Login Screen** state, where the user submits login credentials. During authentication, the system enters the **Authenticating** state.

If the credentials are invalid, the system moves to the **Login Failed** state, allowing the user to retry login or exit the application. If the credentials are valid, the system transitions to either the **Student Authenticated** or **Admin Authenticated** state based on the user role.

After authentication, the system enters the **Student Dashboard** or **Admin Dashboard** state. In the student dashboard, the system can transition between states such as **Viewing Attendance**, **Viewing Marks**, **GPA Calculation**, and **Viewing Report**. In the admin dashboard, the system transitions through states such as **Managing Students**, **Updating Attendance**, **Updating Marks**, and **Viewing All Reports**.

When the user chooses to log out, the system enters the **Logout** state and finally returns to the **Login Screen**, completing the session lifecycle.

This State Diagram ensures controlled navigation, proper session handling, and clear role-based state transitions.



4. PROJECT FLOW

The Smart Semester Progress Tracker follows a well-defined and structured execution flow that ensures smooth interaction between the user, system logic, graphical user interface, and database. Each module is connected logically, allowing users to perform academic tasks step by step without confusion. This section explains how the system operates from application launch to report generation and termination.

4.1 Application Initialization Flow

The project flow begins when the user launches the Smart Semester Progress Tracker application. At startup, the system initializes all required Java classes, establishes database connectivity using JDBC, and loads the graphical user interface components.

Once initialization is completed successfully, the **Welcome Screen** is displayed, providing options to access either the **Student Panel** or the **Admin Panel**.

This step ensures that the system is fully prepared before allowing any user interaction.

4.2 Authentication and Login Flow

When a user selects a panel, the system redirects them to the respective login screen.

4.2.1 Student Login Flow:
The student enters registration number and password. The system validates the credentials by querying the MySQL database. Upon successful authentication, access to the Student Dashboard is granted.

4.2.2 Admin Login Flow:
The administrator enters secure login credentials. After verification, the Admin Dashboard is loaded, allowing the admin to view registered student records stored in the database.

Invalid credentials result in error messages, ensuring secure access control.

4.3 Student Academic Management Flow

After successful login, the student is redirected to the **Student Dashboard**, which acts as the central control panel for all academic activities.

From the dashboard, the student can:

- Register theory and lab courses
- Update or delete registered courses
- Mark attendance for each course
- Enter marks for quizzes, assignments, midterms, and final exams

Each action is performed through a **dedicated GUI screen**. When the student submits any data, the system validates the input and saves it permanently in the database. This ensures accurate record keeping and real-time updates.

4.4 GPA Calculation and Academic Evaluation Flow

Once attendance and marks are entered, the system automatically calculates:

- Attendance percentages
- Total marks per course
- Quality points
- Semester GPA
- Final semester grade

The GPA calculation module applies predefined academic rules and credit-hour weighting. Calculated results are immediately reflected on the dashboard, allowing students to monitor their semester performance at any time.

4.5 Report Generation, Saving, and Printing Flow

The **Report Generation module** allows students to generate comprehensive academic reports based on stored data.

The user can:

- Select report type (Full Academic Report, Course-wise, Category-wise)
- Preview the generated report on-screen
- Save the report as a **Text (.txt) file**
- Print the report directly

For PDF generation, the system uses the operating system's printing functionality. When the user selects **Print Report**, the standard print dialog opens. By choosing **Microsoft Print to PDF**, the report is saved as a **PDF file on the local system**. This approach ensures compatibility without requiring additional libraries.

This flow provides flexibility for documentation, record keeping, and official academic use.

4.6 Admin Monitoring Flow

The Admin Dashboard provides a read-only monitoring interface. The administrator can:

- View registered students
- View academic records stored in the database
- Refresh and monitor system data

The admin does not modify student data, ensuring data integrity and controlled system supervision.

4.7 Data Storage and Retrieval Flow

Throughout the project flow, all academic and user-related data is stored in the MySQL database. The system performs secure read and write operations using JDBC. Proper exception handling ensures that database errors are handled gracefully without system crashes.

This flow guarantees persistent storage and reliable retrieval of academic information.

4.8 Logout and Application Termination Flow

When the user selects the logout option, the system safely terminates the current session and redirects the user back to the Welcome Screen. If the application is closed, all open resources

are released properly while ensuring that stored data remains secure and intact for future sessions.

5. GRAPHICAL USER INTERFACE (GUI) DESCRIPTION

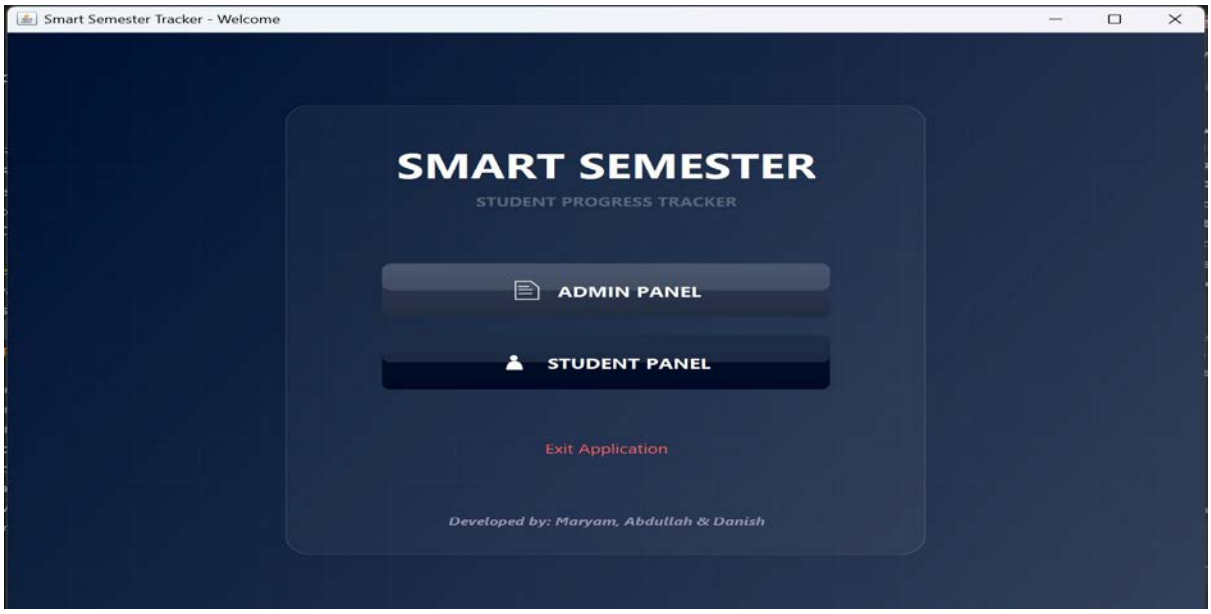
The Graphical User Interface (GUI) of the **Smart Semester Progress Tracker** has been designed with a strong focus on usability, clarity, and visual consistency. The interface enables students and administrators to interact with the system easily without requiring technical knowledge. Each screen is purpose-driven and guides the user step by step through academic management tasks such as course registration, attendance tracking, marks entry, GPA calculation, and report generation.

The GUI follows a clean and modern layout with clearly labeled buttons, input fields, and navigation panels. A consistent color theme and layout structure is maintained across all screens to enhance user experience and ensure smooth navigation between different modules. Proper validation messages, confirmation prompts, and system feedback are provided to reduce user errors and improve interaction efficiency.

The following section explains each GUI screen of the Smart Semester Progress Tracker in detail.

5.1 Welcome / Main Screen

The Welcome Screen is the first screen displayed when the application is launched. It acts as the main entry point of the system and provides users with role-based access options. The screen displays the project title and presents two primary navigation buttons: **Student Panel** and **Admin Panel**.



The **Student Panel** option allows students to proceed to the login process and access academic features such as course management, attendance monitoring, marks entry, GPA calculation, and report generation. The **Admin Panel** option is reserved for administrative access, enabling

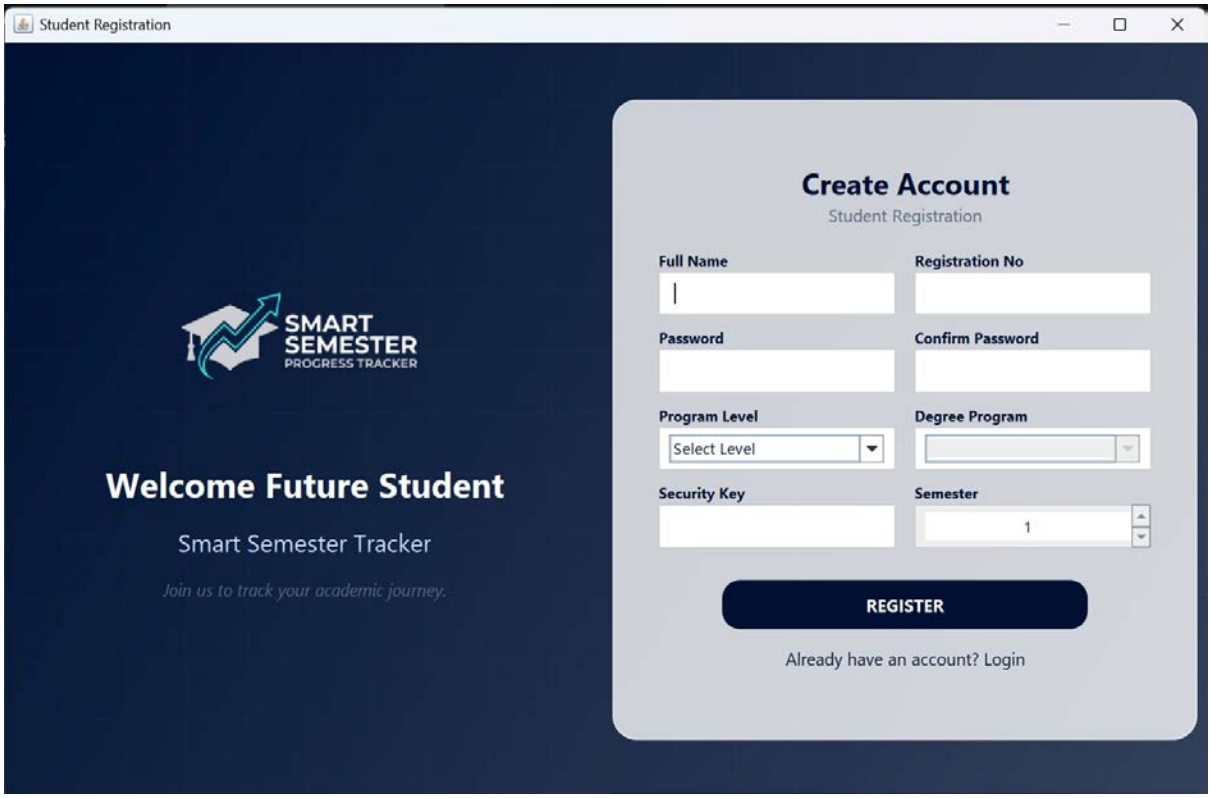
system monitoring and viewing of student records. An **Exit Application** option is also provided to safely close the application.

This screen ensures controlled access, clear role separation, and a professional introduction to the system.

5.3 Student Registration Screen

The Student Registration Screen is designed to allow new students to create an account in the Smart Semester Progress Tracker system. This screen is accessed through the **Sign Up** option available on the Student Login Screen and serves as the initial step for first-time users.

The registration form collects essential student information, including **Full Name**, **Registration Number**, **Password**, and **Confirm Password** to ensure secure account creation. Additional academic details such as **Program Level**, **Degree Program**, and **Current Semester** are also required to correctly map the student's academic profile within the system. A **Security Key** field is included to add an extra layer of verification during the registration process.



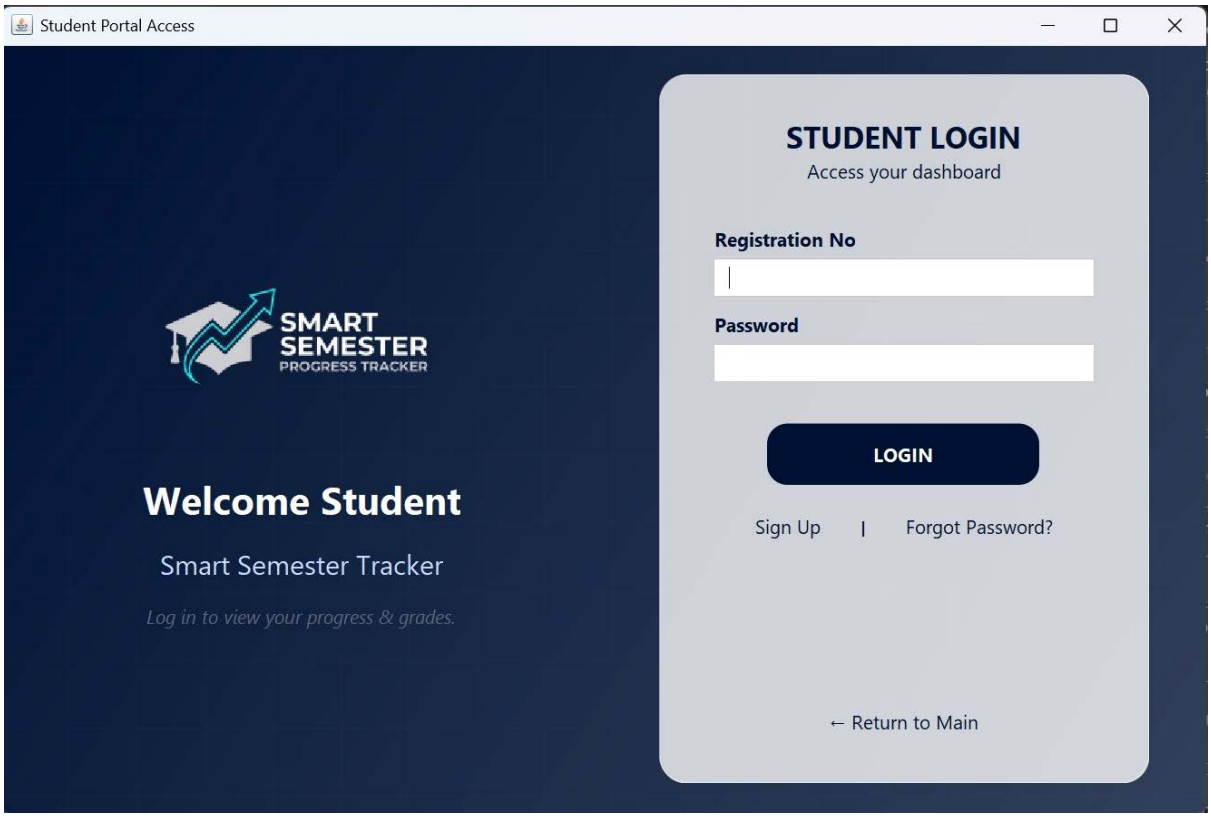
Input validation is applied to ensure that all mandatory fields are filled correctly and that the password and confirmation password match. If invalid or incomplete data is entered, the system provides clear error messages to guide the user. Upon successful registration, the student's data is securely stored in the database, and the user can proceed to log in using the newly created credentials.

An option to navigate back to the login screen is also provided for users who already have an existing account. This screen ensures accurate data entry, secure account creation, and a smooth onboarding experience for students.

5.2 Student Login Screen

The Student Login Screen allows registered students to securely access the Smart Semester Progress Tracker system. This screen is displayed when a user selects the **Student Panel** option from the Welcome Screen. It serves as an authentication gateway to ensure that only authorized students can access their academic data.

The screen contains input fields for **Registration Number** and **Password**, which are validated against the records stored in the database. Proper validation is applied to ensure that empty or incorrect credentials are not accepted. If invalid login details are entered, the system displays an appropriate error message to guide the user.



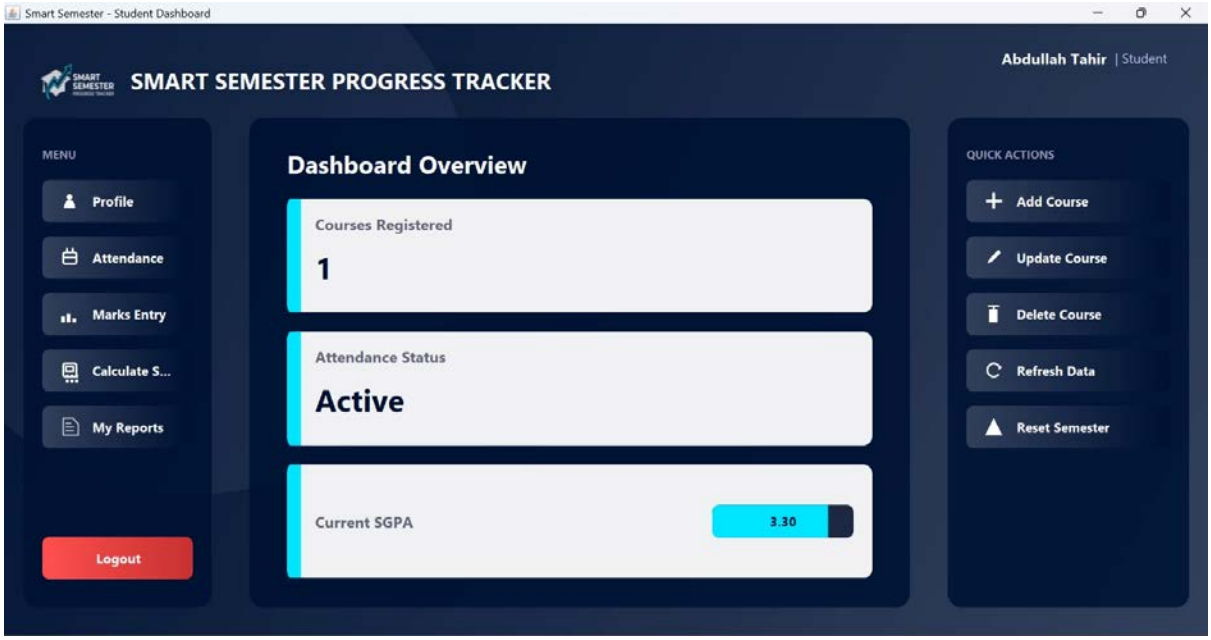
Additional options such as **Sign Up** and **Forgot Password** are provided to support new student registration and password recovery. A **Return to Main** option allows users to navigate back to the Welcome Screen without logging in.

Once the login is successful, the student is redirected to the **Student Dashboard**, where they can manage courses, track attendance, enter marks, calculate GPA, and generate academic reports. This screen ensures secure access, user convenience, and smooth navigation into the system.

5.4 Student Dashboard Screen

The Student Dashboard Screen is displayed immediately after a successful student login. It serves as the central control panel for students, providing a summarized overview of their academic status and quick access to all major functionalities of the Smart Semester Progress Tracker.

The dashboard presents key academic indicators such as the **number of courses registered**, **attendance status**, and **current SGPA**, allowing students to quickly assess their semester progress at a glance. These indicators are dynamically updated based on the data stored in the database.



A navigation menu is provided on the left side of the screen, enabling students to access different modules including **Profile**, **Attendance**, **Marks Entry**, **GPA Calculation**, and **My Reports**. On the right side, a **Quick Actions** panel is available, offering shortcuts for frequently used operations such as **Add Course**, **Update Course**, **Delete Course**, **Refresh Data**, and **Reset Semester**.

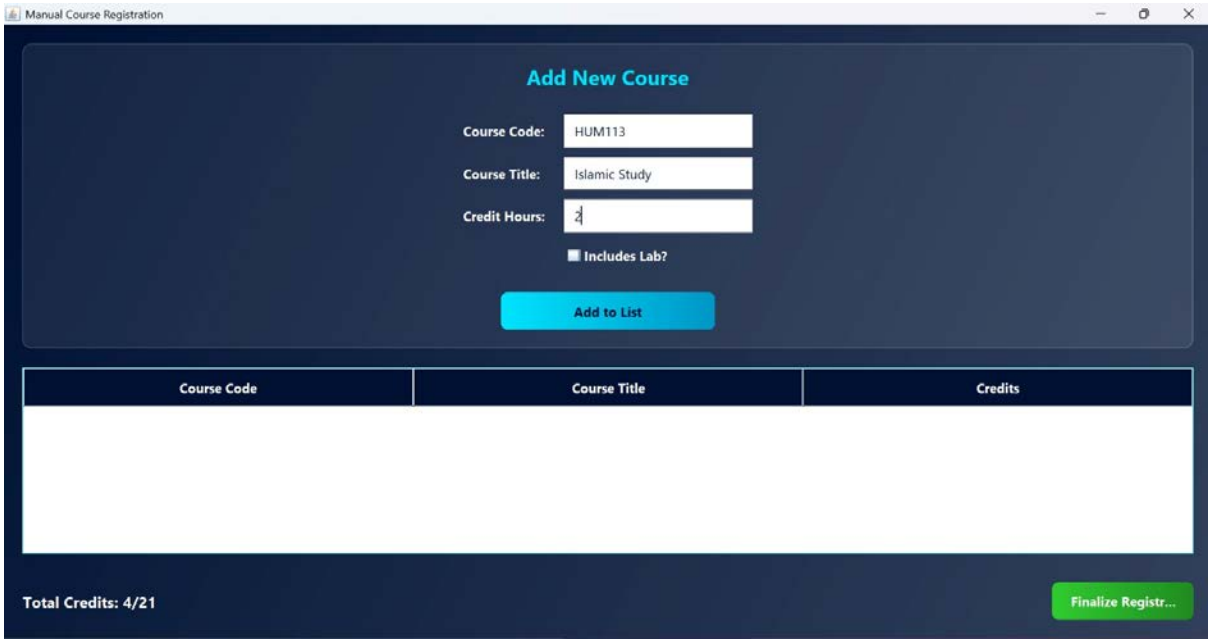
The dashboard layout is clean, well-organized, and visually consistent with the overall system design. A **Logout** option is included to allow secure termination of the session. This screen enhances usability by centralizing academic controls and ensuring smooth navigation across the system.

5.5 Course Registration (Add New Course) Screen

The Course Registration Screen allows students to manually add and register courses for the current semester. This screen provides a structured interface for entering complete course details and managing semester course enrollment.

Students can enter the **Course Code**, **Course Title**, and **Credit Hours** for each course. An additional option, **Includes Lab**, is provided to specify whether the course contains a lab

component. Upon entering the required information, the **Add to List** button adds the course to the course table displayed on the same screen.



The lower section of the screen displays a tabular list of all added courses, showing details such as course code, course title, and credit hours. A **Total Credits** indicator is also displayed to help students monitor their credit load and ensure it remains within the allowed semester limit.

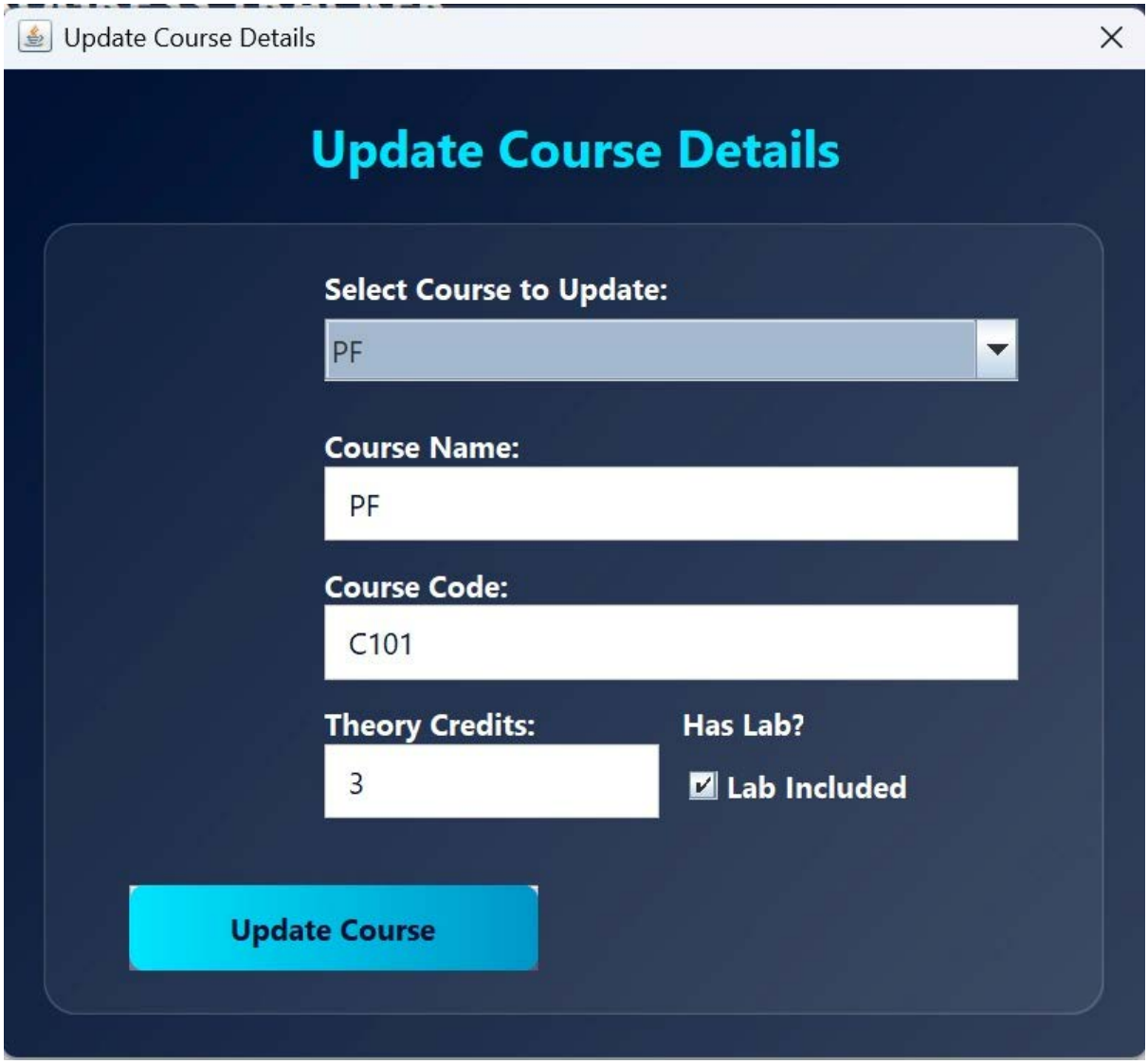
Once all desired courses are added, the **Finalize Registration** option allows students to confirm and save their course registration. All registered course data is securely stored in the database and becomes available for attendance tracking, marks entry, GPA calculation, and report generation.

This screen ensures organized course management, prevents credit overload, and supports accurate academic record keeping.

5.6 Update Course Details Screen

The Update Course Details screen is designed to allow students to revise information of courses that have already been registered in the system. Using the drop-down list, the student can select a specific course and update key details such as the **course name**, **course code**, **theory credit hours**, and whether the course **includes a lab component**.

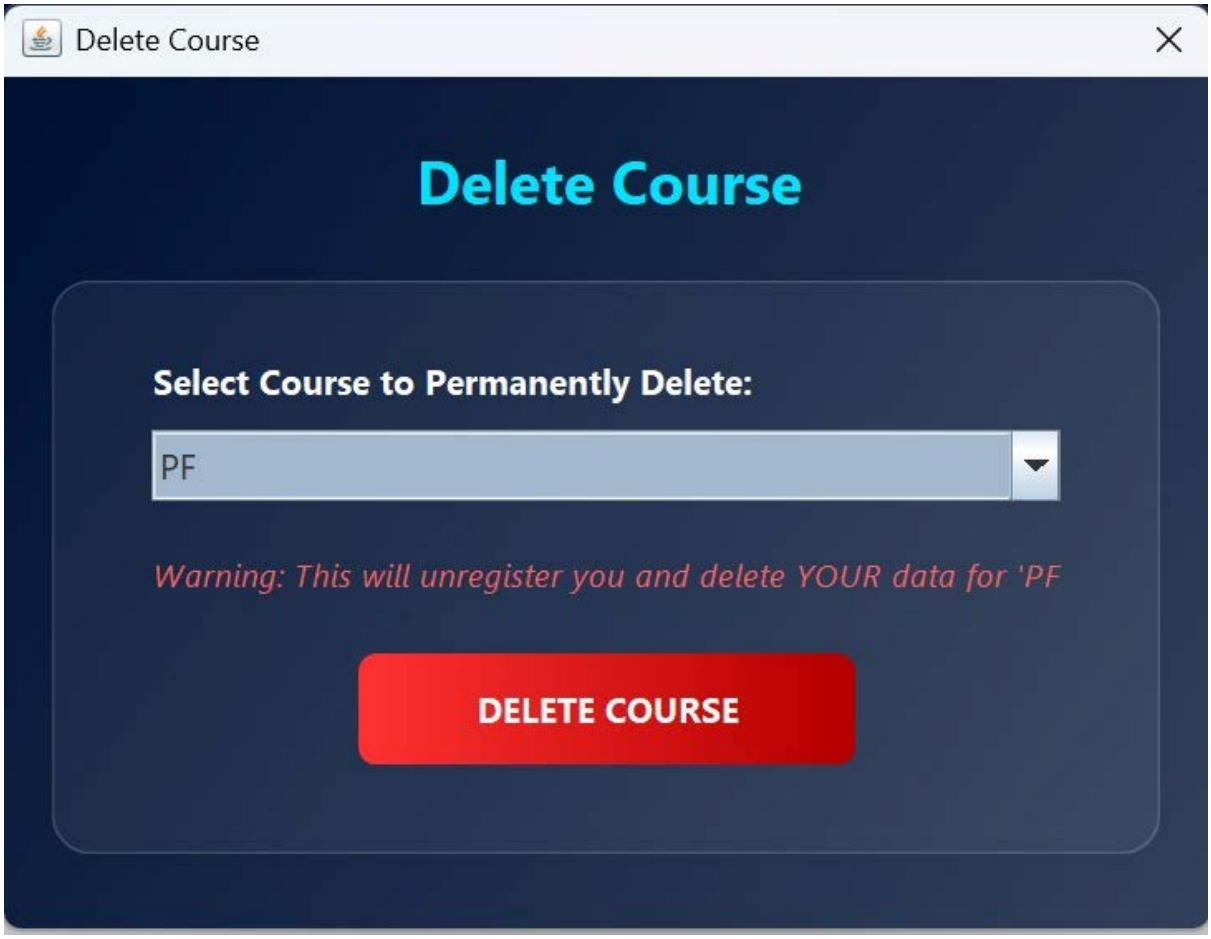
This functionality ensures flexibility in academic management by allowing users to correct mistakes made during course registration or accommodate changes in course structure. Once the update is confirmed, the modified course information is stored in the database and automatically synchronized with other system modules, including attendance records, marks entry, GPA calculation, and academic report generation. This ensures data consistency throughout the application



5.7 Delete Course Screen

The Delete Course screen allows students to permanently remove a registered course from their semester record. The user selects the desired course from the drop-down list and is shown a clear warning message before deletion to prevent accidental data loss.

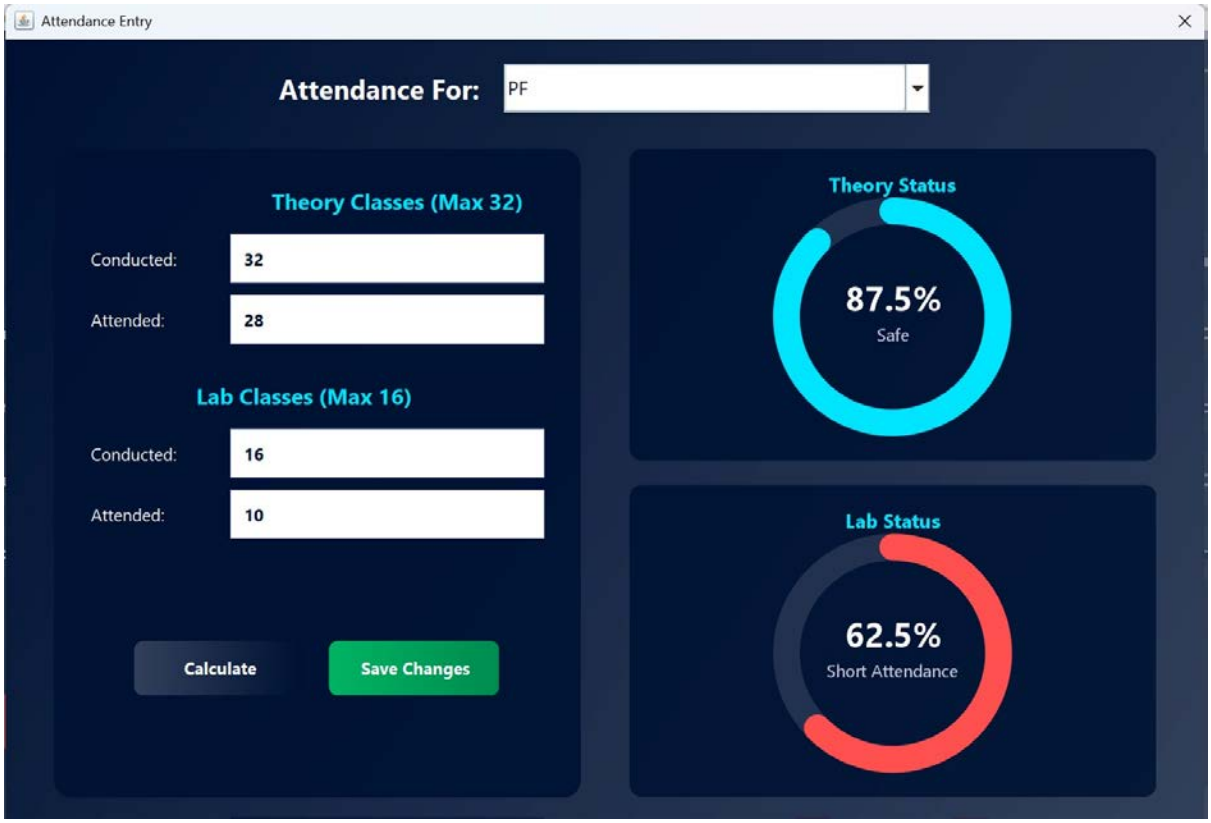
Once the delete action is confirmed, the selected course is unregistered from the student's profile, and all related data such as **attendance records, marks, GPA calculations, and reports** for that course are removed from the database. This feature ensures accurate semester management and allows students to correct registration decisions when necessary.



5.8 Attendance Entry Screen

The Attendance Entry screen allows students to record and monitor their attendance for each registered course. The user first selects a course, after which attendance details for both **theory classes** and **lab classes** can be entered separately by specifying the number of **classes conducted** and **classes attended**.

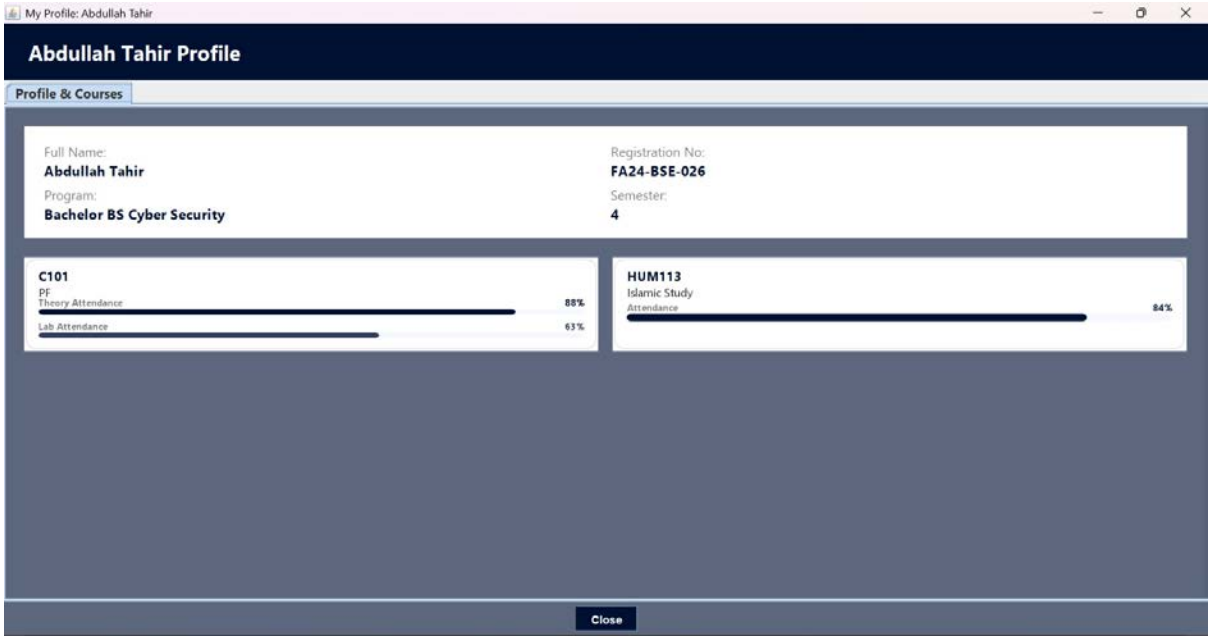
The system automatically calculates attendance percentages and visually displays the **attendance status** using graphical indicators. A “Safe” status is shown when attendance meets the required criteria, while a “Short Attendance” warning is displayed if attendance falls below the acceptable limit. This feature helps students continuously track their attendance and take timely corrective actions. All changes can be saved securely to the database for future reference.



5.9 Student Profile and Course Overview Screen

The Student Profile screen displays complete academic and personal information of the logged-in student in a clear and organized manner. It shows key details such as full name, registration number, degree program, and current semester, providing a quick overview of the student's profile.

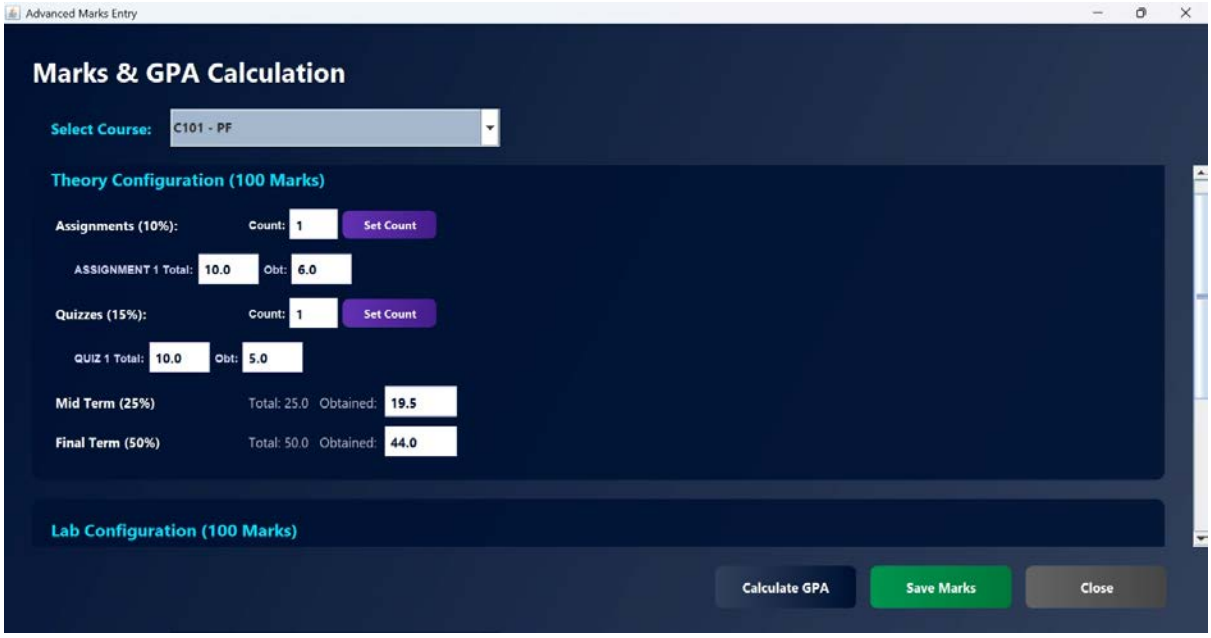
Below the profile information, the screen presents a summary of all registered courses along with their attendance status. Separate progress bars are used to visually represent theory attendance and lab attendance for each course, allowing students to easily monitor their attendance performance. This screen helps students track their academic progress at a glance and ensures transparency in attendance records.

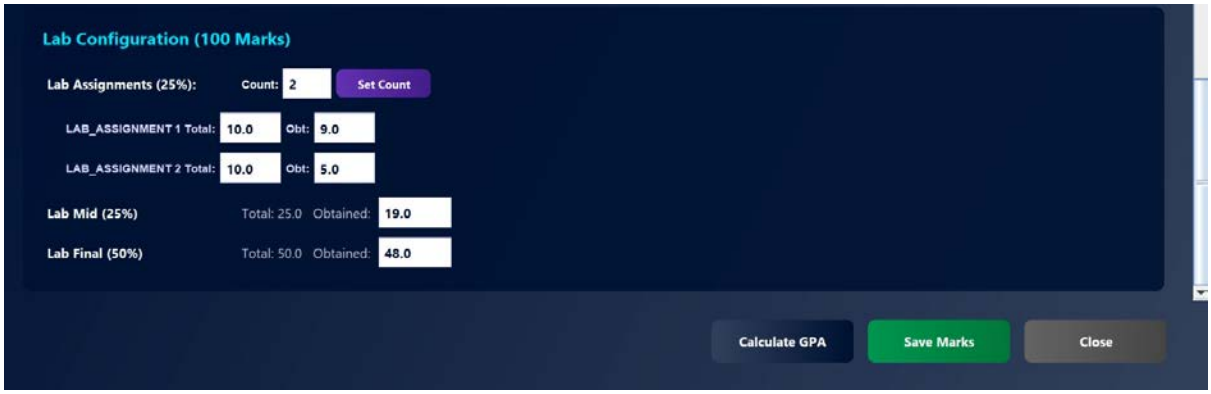


5.10 Marks Entry and GPA Calculation Screen

The Marks Entry and GPA Calculation screen allows students to enter and manage academic marks for each registered course. After selecting a course, the system provides separate sections for **theory** and **lab** evaluation, each configured with a total of 100 marks.

For theory evaluation, students can enter marks for **assignments**, **quizzes**, **mid-term**, and **final examinations** according to their respective weightages. The system also allows dynamic configuration of the number of assignments and quizzes. Similarly, the lab section enables entry of **lab assignments**, **lab mid-term**, and **lab final exam** marks.





Based on the entered data, the system automatically calculates the **weighted total**, **final score**, **grade**, and **GPA** for the selected course. All marks can be saved securely in the database and are used in semester GPA calculation and report generation, ensuring accuracy and consistency across the system.

5.11 Detailed Result Screen

The **Detailed Result Screen** provides a complete and structured breakdown of a student's academic performance for a selected course. This screen is divided into **Theory Result** and **Lab Result** tabs, allowing students to view marks category-wise in a clear and organized manner.

Each section displays assessment components such as **assignments, quizzes, mid-term, and final exam**, along with **obtained marks, total marks, and percentage** for every component. This detailed view helps students understand their strengths and weaknesses in each assessment area.

At the bottom of the screen, the system automatically calculates and displays the **final GPA, total marks, overall percentage, and course grade**, providing a summarized performance overview. The clean tab-based layout improves readability and allows smooth switching between theory and lab results without confusion.

Assessment	Obtained	Total	Percentage
ASSIGNMENT 1	9.0	10.0	90.0%
ASSIGNMENT 2	7.0	10.0	70.0%
QUIZ 1	7.0	10.0	70.0%
QUIZ 2	4.0	5.0	80.0%
QUIZ 3	4.0	10.0	40.0%
Mid Term	20.0	25.0	80.0%
Final Exam	40.5	50.0	81.0%

5.12 GPA Calculation Result Screen

The **GPA Calculation Result Screen** displays the final evaluated performance of a student for a selected course after all marks have been entered and processed. This screen clearly shows the **weighted theory total**, **weighted lab total**, and the **final combined score** based on the predefined weightage (e.g., 75% Theory and 25% Lab).

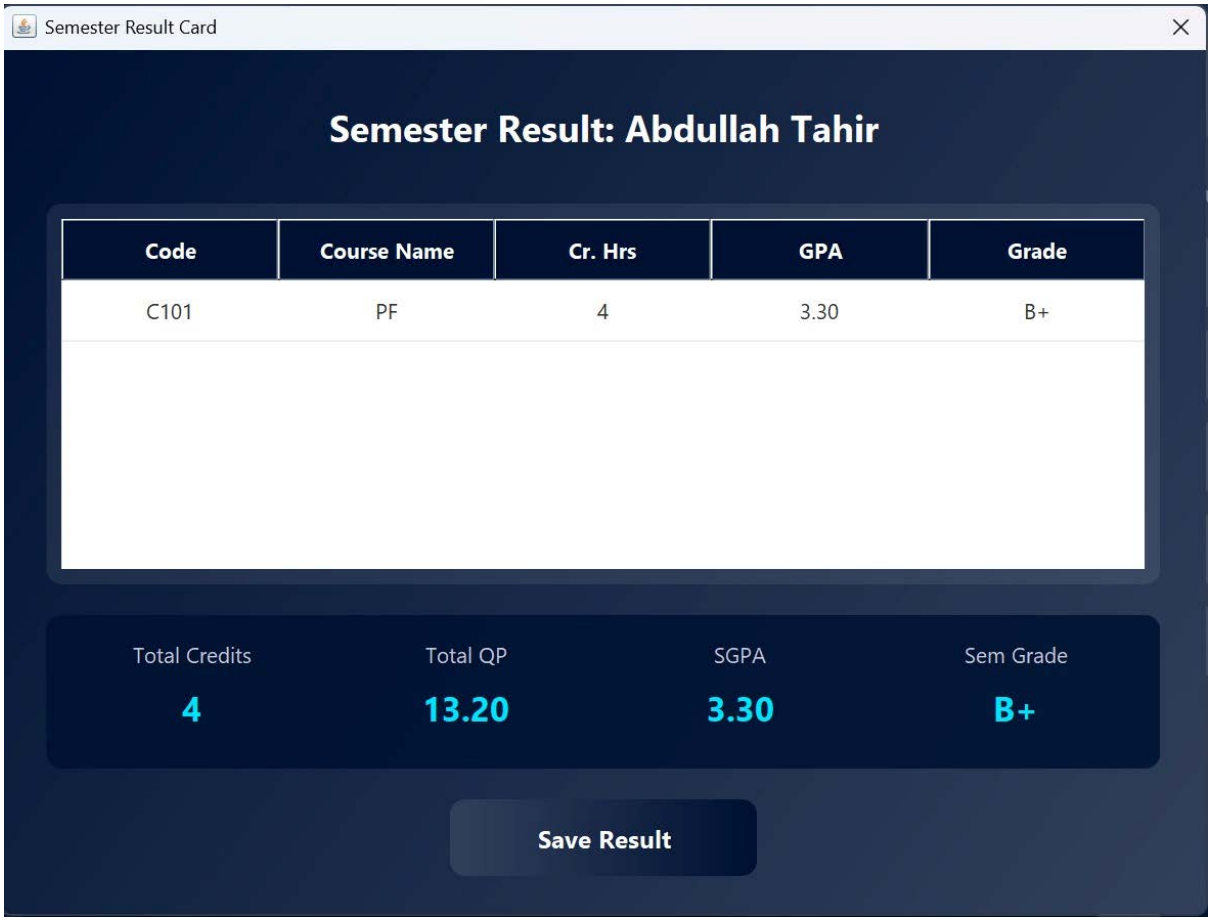
Based on the final score, the system automatically determines and displays the **course grade** and the corresponding **GPA** according to the grading policy. This ensures accurate, transparent, and standardized result calculation without any manual effort.

The screen serves as a confirmation view for students, allowing them to quickly understand their overall performance before proceeding further. The simple layout and highlighted GPA and grade improve clarity and make the result easy to interpret.

5.13 Semester Result Card Screen

The **Semester Result Card Screen** presents a summarized view of the student's overall academic performance for the completed semester. This screen displays a tabular list of all registered courses, including **course code, course name, credit hours, GPA, and grade** for each course.

Below the course table, the system automatically calculates and highlights key semester statistics such as **total registered credits**, **total quality points (QP)**, **Semester GPA (SGPA)**, and the **overall semester grade**. These values are computed based on standard GPA calculation rules, ensuring accuracy and consistency.



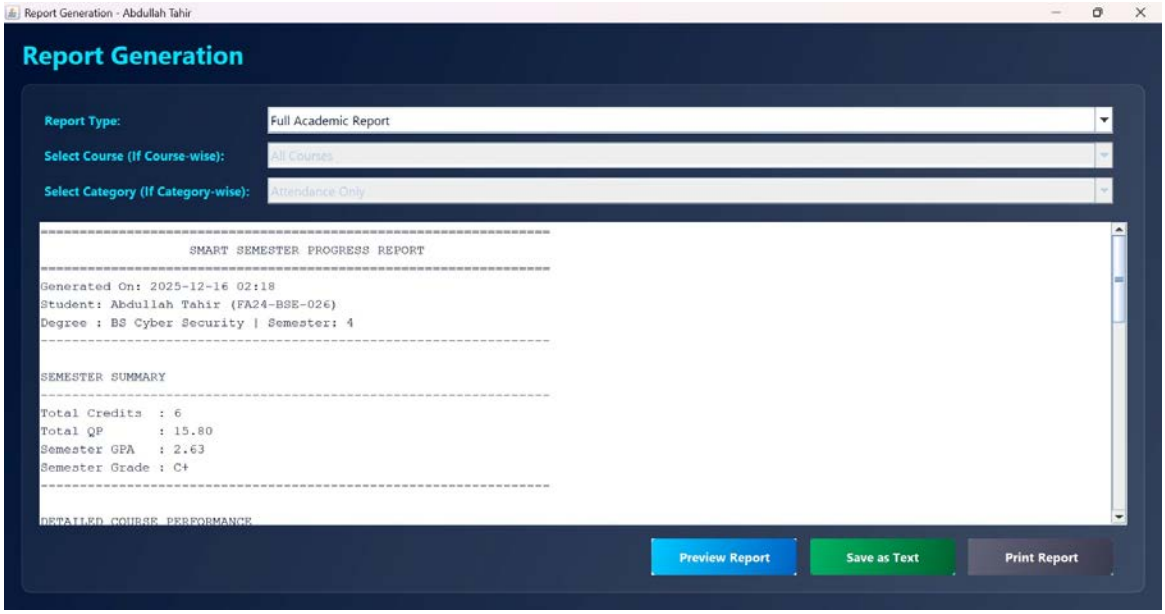
The **Save Result** option allows the finalized semester result to be stored permanently in the database for future reference and report generation. This screen provides a clear, concise, and official representation of the student's semester performance, similar to a traditional university result card.

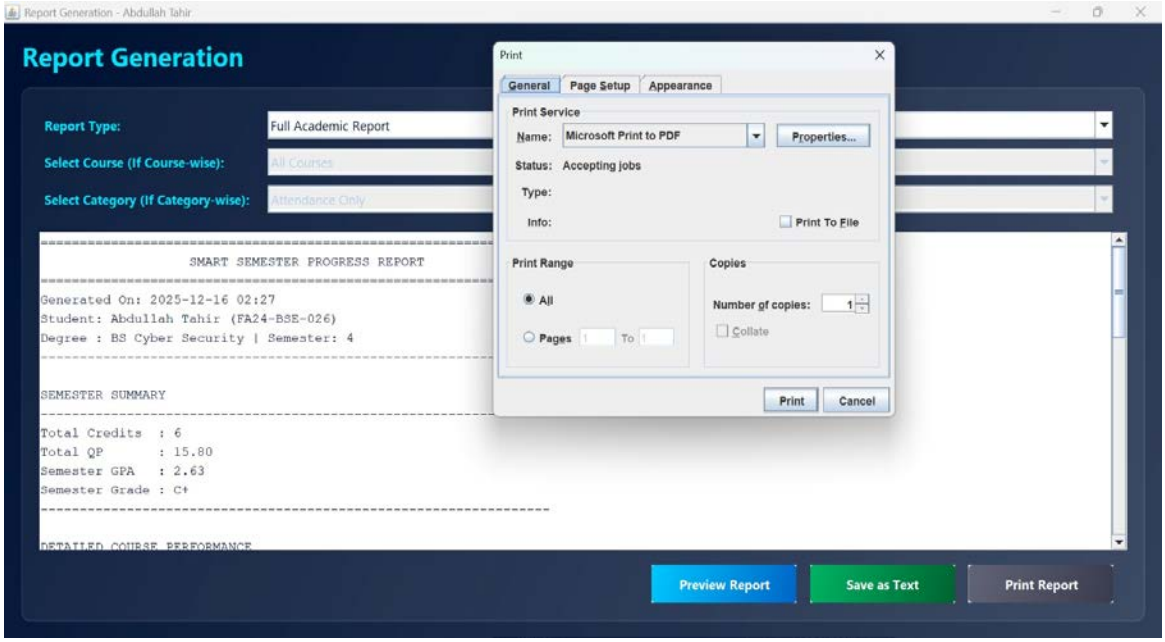
5.14 Report Generation and PDF Export

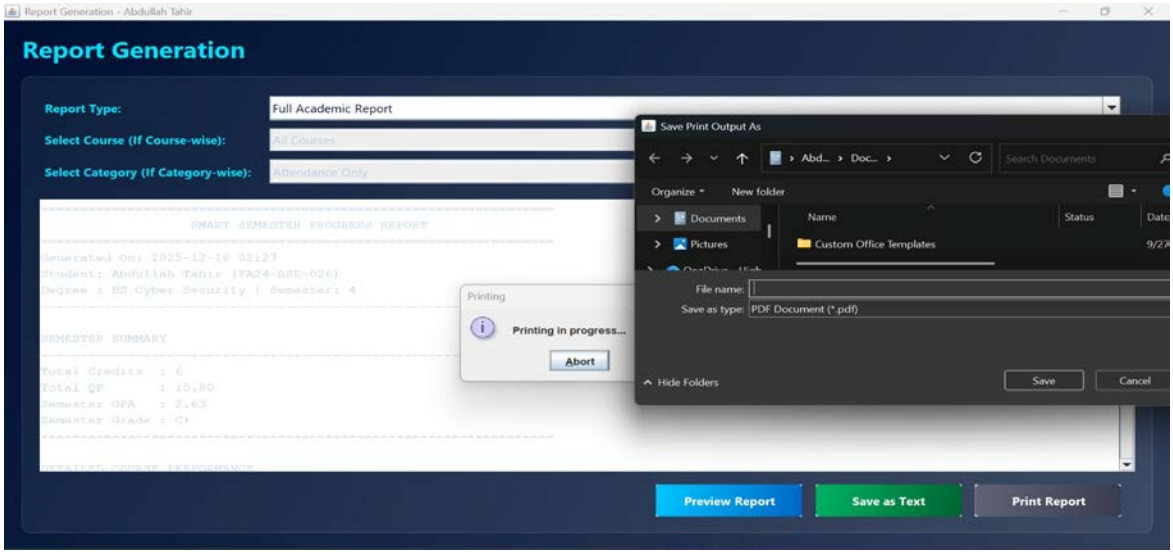
The Report Generation Screen allows students to generate comprehensive academic reports based on selected criteria. Users can choose the report type and apply course-wise or category-wise filters such as attendance or marks, and instantly preview the generated academic report within the system.

The report includes essential academic details such as student information, degree program, semester details, semester summary, GPA, grades, and detailed course performance. All report data is retrieved directly from the database, ensuring accuracy, consistency, and reliability.

The system provides multiple output options, including previewing the report, saving it as a text file, or exporting it as a PDF document using the print-to-PDF functionality. This feature enables students to securely store, share, or submit their academic records in a well-structured and professional format.


Report Generation - Abdullah Tahir
Report Generation
Report Type:
Full Academic Report
Select Course (If Course-wise):
All Courses
Select Category (If Category-wise):
Attendance Only
SMART SEMESTER PROGRESS REPORT
Generated On: 2025-12-16 02:18
Student: Abdullah Tahir (FA24-BSE-026)
Degree : BS Cyber Security | Semester: 4
SEMESTER SUMMARY
Total Credits : 6
Total QP : 15.80
Semester GPA : 2.63
Semester Grade : C+
DETAILED COURSE PERFORMANCE
Preview Report
Save as Text
Print Report


Report Generation - Abdullah Tahir
Report Generation
Report Type:
Full Academic Report
Select Course (If Course-wise):
All Courses
Select Category (If Category-wise):
Attendance Only
SMART SEMESTER PROGRESS REPORT
Generated On: 2025-12-16 02:27
Student: Abdullah Tahir (FA24-BSE-026)
Degree : BS Cyber Security | Semester: 4
SEMESTER SUMMARY
Total Credits : 6
Total QP : 15.80
Semester GPA : 2.63
Semester Grade : C+
DETAILED COURSE PERFORMANCE
Print
General
Page Setup
Appearance
Print Service
Name:
Microsoft Print to PDF
Properties...
Status:
Accepting jobs
Type:
Info:
Print To File
Print Range
All
Pages
To
Copies
Number of copies:
Collate
Print
Cancel
Preview Report
Save as Text
Print Report

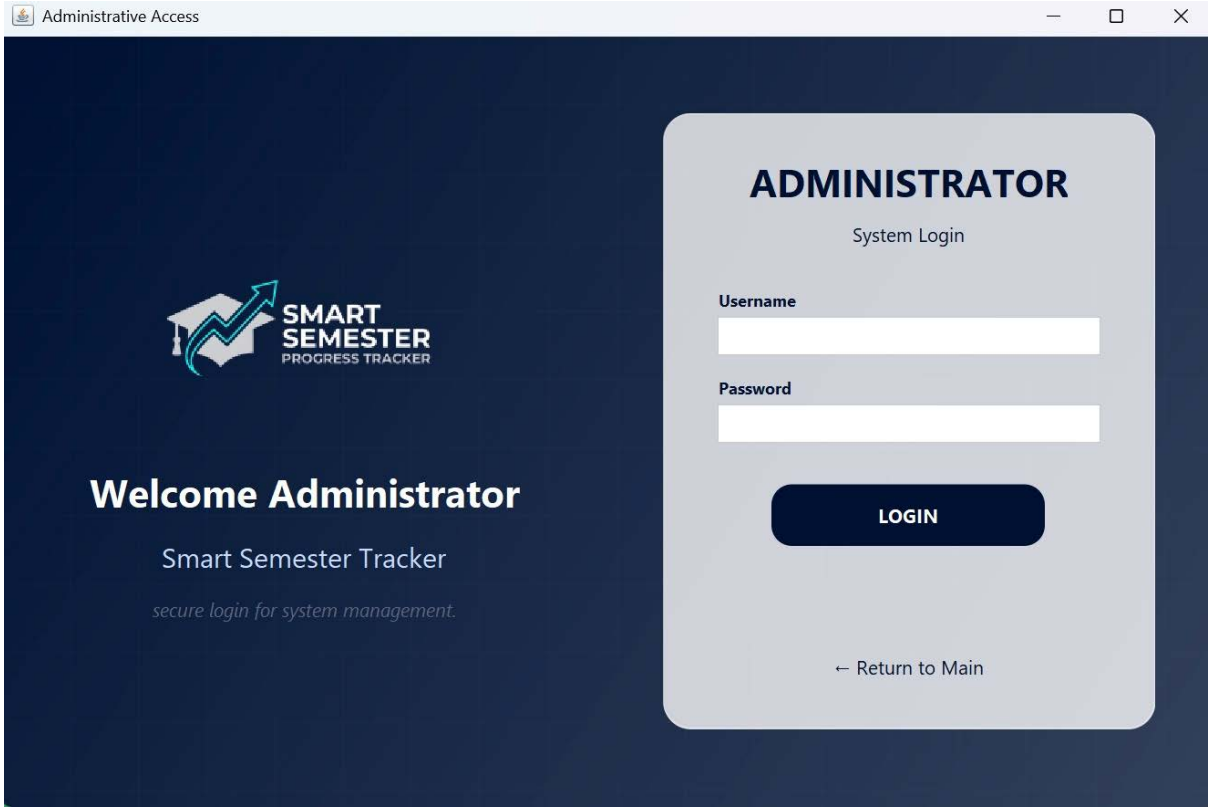

Report Generation - Abdullah Tahir
Report Generation
Report Type:
Full Academic Report
Select Course (If Course-wise):
Select Category (If Category-wise):
Printing
Printing in progress...
Abort
Save Print Output As
Organize
New folder
Documents
Pictures
Name
Status
Custom Office Templates
File name:
Save as type:
PDF Document (*.pdf)
Hide Folders
Save
Cancel
Preview Report
Save as Text
Print Report

5.15 Administrator Login Screen

The **Administrator Login** screen provides secure access to the administrative section of the Smart Semester Progress Tracker. This screen is specifically designed for system administrators who are responsible for managing and monitoring overall system data.

The administrator is required to enter a valid **username** and **password** to gain access. Authentication ensures that only authorized personnel can access sensitive information such as student records, course data, and database-level operations. The screen also includes a **Return to Main** option, allowing the administrator to safely navigate back to the main welcome screen without logging in.

This module enhances system security by separating **administrative privileges** from regular student access and ensures controlled system management.



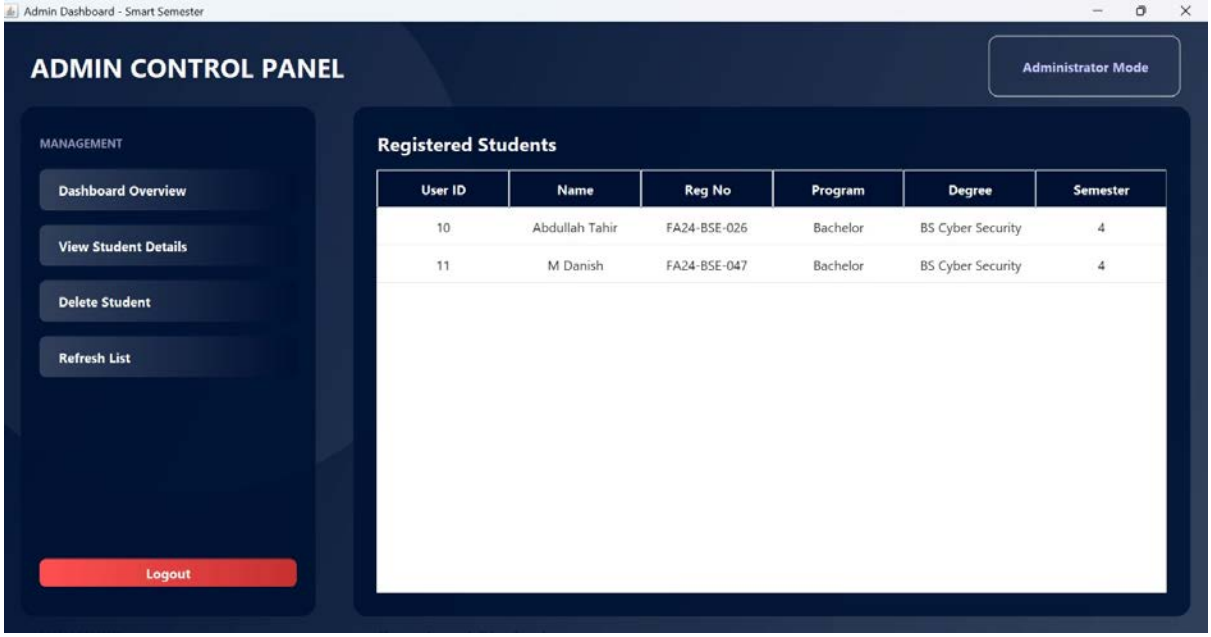
5.16 Admin Control Panel (Admin Dashboard)

The **Admin Control Panel** serves as the central management interface for administrators in the Smart Semester Progress Tracker. After successful login, the administrator is directed to this dashboard, which provides full visibility and control over registered students within the system.

The dashboard displays a **Registered Students table** containing essential information such as **User ID, Student Name, Registration Number, Program Level, Degree Program, and Current Semester**. This allows the administrator to easily monitor and verify student records stored in the database.

On the left-side **Management Menu**, the administrator can perform key actions including **viewing student details**, **deleting student records**, and **refreshing the student list** to reflect real-time database updates. A **Logout** option is also provided to ensure secure session termination.

This module ensures effective **administrative control, data integrity, and system-level management**, keeping administrative operations completely separate from student functionalities.

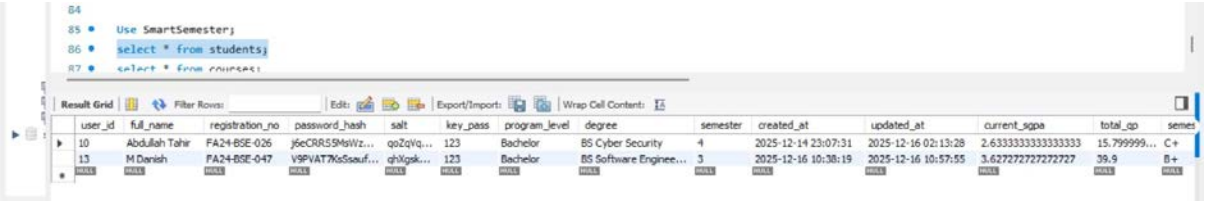


6. Database Queries and Tables

6.1 Student Table and Query

```
CREATE TABLE IF NOT EXISTS students (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(255) NOT NULL,
    registration_no VARCHAR(50) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    salt VARCHAR(255) NOT NULL,
    key_pass VARCHAR(255),
    program_level VARCHAR(50),
    degree VARCHAR(100),
```

```
    semester INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    INDEX idx_registration_no (registration_no)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
Use SmartSemester;
select * from students;
```



6.2 Course Table and Query

```
-- course table
CREATE TABLE IF NOT EXISTS courses (
    course_code VARCHAR(20) PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    theory_credits INT NOT NULL DEFAULT 0,
    lab_credits INT NOT NULL DEFAULT 0,
    has_lab BOOLEAN DEFAULT FALSE,
    is_fyp BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

6.2.1 Insert Table and Query

```
INSERT INTO courses (course_code, title, theory_credits, lab_credits, has_lab, is_fyp) VALUES

 ('CS101', 'Introduction to Computer Science', 3, 0, FALSE, FALSE),

 ('CS201', 'Data Structures', 3, 1, TRUE, FALSE),

('CS301', 'Database Systems', 3, 1, TRUE, FALSE),

('CS401', 'Software Engineering', 3, 0, FALSE, FALSE),

('CS499', 'Final Year Project', 0, 6, FALSE, TRUE)

ON DUPLICATE KEY UPDATE title=VALUES(title);

Use SmartSemester;

select * from courses;
```

```
91 •  select * from courses;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

course_code	title	theory_credits	lab_credits	has_lab	is_fyp	created_at	updated_at
aff123	aafsf	3	0	1	0	2025-12-14 21:36:01	2025-12-14 21:36:01
C101	PF	3	1	1	0	2025-12-12 12:09:18	2025-12-15 09:28:33
C102	OOP	4	0	1	0	2025-12-12 12:09:18	2025-12-14 02:24:00
C103	OOP	3	1	1	0	2025-12-14 12:10:17	2025-12-14 12:32:48
C105	DATABASS	4	0	0	0	2025-12-12 12:09:18	2025-12-12 12:09:18
C106	SE	3	0	0	0	2025-12-12 12:09:18	2025-12-12 12:09:18
C111	OOP	4	1	1	0	2025-12-12 13:07:22	2025-12-16 10:42:58
C119	AI	4	0	1	0	2025-12-13 17:57:37	2025-12-13 17:57:37
CC102	DS	4	0	0	0	2025-12-13 00:41:57	2025-12-13 18:26:32
CS101	Introduction to Computer Science	3	0	0	0	2025-12-10 23:06:24	2025-12-10 23:06:24
CS201	Data Structures	3	1	1	0	2025-12-10 23:06:24	2025-12-10 23:06:24
CS301	Database Systems	3	1	1	0	2025-12-10 23:06:24	2025-12-10 23:06:24
CS401	Software Engineering	3	0	0	0	2025-12-10 23:06:24	2025-12-10 23:06:24
CS499	Final Year Project	0	6	0	1	2025-12-10 23:06:24	2025-12-10 23:06:24

6.3 Course Registration Table and Query

```
-- Course Registrations table (links students to courses)

CREATE TABLE IF NOT EXISTS course_registrations (

    registration_id INT AUTO_INCREMENT PRIMARY KEY,

    student_id INT NOT NULL,

    course_code VARCHAR(20) NOT NULL,

    registered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (student_id) REFERENCES students(user_id) ON DELETE CASCADE,
```

```
    FOREIGN KEY (course_code) REFERENCES courses(course_code) ON DELETE CASCADE,

    UNIQUE KEY unique_student_course (student_id, course_code),

    INDEX idx_student_id (student_id),

    INDEX idx_course_code (course_code)

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

Use SmartSemester;

select * from course_registrations;
```

```
90 •  select * from course_registrations;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

registration_id	student_id	course_code	registered_at	final_grade	gpa
29	10	C101	2025-12-15 16:49:03	C+	2.3
30	10	HUM113	2025-12-15 20:46:03	B+	3.3
31	13	C101	2025-12-16 05:42:58	A-	3.7
32	13	HUM113	2025-12-16 05:42:58	B+	3.3
33	13	C111	2025-12-16 05:42:58	A-	3.7
NULL	NULL	NULL	NULL	NULL	NULL

6.4 Attendance Table and Query

```
-- Attendance Records table

CREATE TABLE IF NOT EXISTS attendance_records (

    attendance_id INT AUTO_INCREMENT PRIMARY KEY,

    registration_id INT NOT NULL,

    mid_conducted INT DEFAULT 0,

    mid_attended INT DEFAULT 0,

    final_conducted INT DEFAULT 0,

    final_attended INT DEFAULT 0,

    lab_conducted_mid INT DEFAULT 0,

    lab_attended_mid INT DEFAULT 0,

    lab_conducted_final INT DEFAULT 0,

    lab_attended_final INT DEFAULT 0,
```

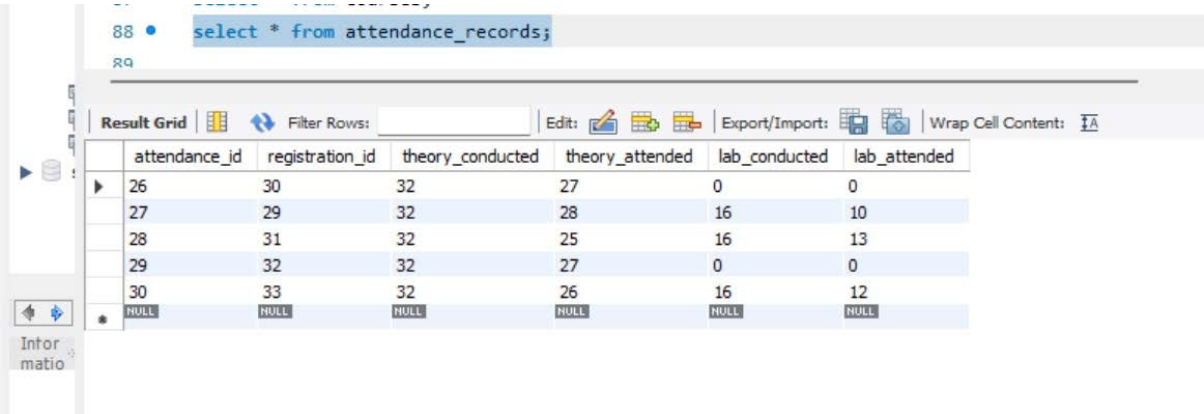
```
    FOREIGN KEY (registration_id) REFERENCES course_registrations(registration_id) ON DELETE CASCADE,

    UNIQUE KEY unique_registration_attendance (registration_id)

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

Use SmartSemester;

select * from attendance_records;
```



attendance_id	registration_id	theory_conducted	theory_attended	lab_conducted	lab_attended
26	30	32	27	0	0
27	29	32	28	16	10
28	31	32	25	16	13
29	32	32	27	0	0
30	33	32	26	16	12
NULL	NULL	NULL	NULL	NULL	NULL

6.5 Marks Table and Query

```
 -- Marks table

CREATE TABLE IF NOT EXISTS marks (

    mark_id INT AUTO_INCREMENT PRIMARY KEY,

    registration_id INT NOT NULL,

    assessment_type ENUM('QUIZ', 'ASSIGNMENT', 'MID', 'FINAL', 'LAB_ASSIGNMENT', 'LAB_MID', 'LAB_FINAL') NOT NULL,

    obtained DOUBLE NOT NULL,

    graded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (registration_id) REFERENCES course_registrations(registration_id) ON DELETE CASCADE,

    INDEX idx_registration_id (registration_id),

    INDEX idx_assessment_type (assessment_type)

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

Use SmartSemester;
```

```
select * from marks;
```

```
89 •  select * from marks;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

mark_id	registration_id	assessment_type	obtained	graded_at	total_marks	assessment_label
121	30	MID	20	2025-12-15 21:07:35	25	Mid Term (25%)
122	30	FINAL	40.5	2025-12-15 21:07:35	50	Final Term (50%)
123	30	ASSIGNMENT	9	2025-12-15 21:07:35	10	ASSIGNMENT 1
124	30	ASSIGNMENT	7	2025-12-15 21:07:35	10	ASSIGNMENT 2
125	30	QUIZ	7	2025-12-15 21:07:35	10	QUIZ 1
126	30	QUIZ	4	2025-12-15 21:07:35	5	QUIZ 2
127	30	QUIZ	4	2025-12-15 21:07:35	10	QUIZ 3
146	29	ASSIGNMENT	2	2025-12-16 04:47:27	10	ASSIGNMENT 1
147	29	ASSIGNMENT	4	2025-12-16 04:47:27	10	ASSIGNMENT 2
148	29	QUIZ	5	2025-12-16 04:47:27	10	QUIZ 1
149	29	MID	19.5	2025-12-16 04:47:27	25	Mid Term (25%)
150	29	FINAL	30	2025-12-16 04:47:27	50	Final Term (50%)
151	29	LAB_ASSIGNMENT	9	2025-12-16 04:47:27	10	LAB_ASSIGNME...
152	29	LAB_ASSIGNMENT	5	2025-12-16 04:47:27	10	LAB_ASSIGNME...
153	29	LAB_MID	19	2025-12-16 04:47:27	25	Lab Mid (25%)
154	29	LAB_FINAL	38	2025-12-16 04:47:27	50	Lab Final (50%)
155	31	MID	21	2025-12-16 05:47:23	25	Mid Term (25%)

marks 3

7. Team Contribution Table – Smart Semester Progress Tracker

Member Name	Core Development & Logic	GUI Screens Implemented	Database, Reports & Documentation	Presentation Responsibility
Maryam Zahid (FA24-BSE-035)	Co-developed the Student Module backend logic using Java OOP, including course registration, attendance processing, marks entry flow, GPA calculation support, and	Designed and implemented major Student-side GUI screens such as Student Login, Registration, Dashboard, Course Registration, Attendance Entry, Profile, and Result	Worked on student-related database integration (JDBC), handled insertion and retrieval of attendance and course data, and contributed to student module documentation.	Introduction, OOP Concepts, Welcome Screen, Student Registration, Student Login, Student Dashboard, Course Management, Attendance Module

	semester tracking.	Overview using Java Swing.		
Abdullah Tahir (FA24-BSE-026)	Co-developed the Student Module core logic including advanced marks calculation, weighted assessments, GPA & grade generation, semester summary logic, and report generation processing.	Designed Marks Entry, GPA Calculation, Detailed Result, Semester Result Card, and Report Generation GUI including preview, text export, and PDF print functionality.	Actively worked on student academic database queries, GPA/report data retrieval from MySQL Server, and documented GPA calculation, report generation, and student workflow.	Marks Entry, GPA Calculation, Detailed Result, Semester Result Card, Report Generation & Saving
Muhammad Danish (FA24-BSE-047)	Developed the complete Admin Module backend logic, including student record management, admin controls, and system-level operations.	Designed and implemented the Admin Panel GUI including Admin Login, Admin Dashboard, Student List View, and Admin Actions screens.	Designed the MySQL Server database schema, created tables, relationships, constraints, and handled admin-level database operations and documentation.	Admin Login, Admin Dashboard, Database Design, SQL Tables & Queries, Conclusion

7.1 Smart Semester Progress Tracker Table For Marks

Member Name	**Core Development & Logic**	**GUI Screens Implemented**	**Database, Reports & Documentation**	**Presentation Responsibility**
Maryam Zahid (FA24-BSE-035)				

Abdullah Tahir (FA24-BSE-026)				,
Muhammad Danish (FA24-BSE-047)		.		